

HASHING

PROBLEM 9

QUESTION:

DS	Session	Hashing	Question Information	Level 1	Challenge 99
Problem Description There are N chocolates denoted by array A where $A[i]$ is the length of the i-th chocolate. Alice can melt each chocolate and then convert it into a chocolate whose length is any divisor of the number $A[i]$. So, a chocolate of length $A[i]$ can be converted into X different types of chocolate where X is the count of divisors of the number $A[i]$. So you need to count the total unordered pair of chocolates such that their X value is same. Constraints $1 \leq N \leq 10^5$ $1 \leq A[i] \leq 10^6$ Input Format The first line contains an integer N as input denoting the total number of elements in the array A. The next line contains N space-separated integers that denote the elements of the array A. Output Format In the output, print the total number of ways as mentioned in the statement.					
Logical Test Cases Test Case 1 INPUT (STDIN) 3 2 3 4 EXPECTED OUTPUT 1 Test Case 2 INPUT (STDIN) 10 2 3 4 2 7 6 8 6 4 9 EXPECTED OUTPUT 12					
Mandatory Test Cases Test Case 1 KEYWORD while(--N)					
Complexity Test Cases Test Case 1 CYCLOMATIC COMPLEXITY 8 Test Case 2 TOKEN COUNT 345 Test Case 3 NLOC 44					

SOURCE CODE:

```
#include <stdio.h>

#define MAX_NUM_VAL 1000005

int num_divisors[MAX_NUM_VAL];
long long divisor_tally[1000];

void sieve_divisors() {
    for (int i = 1; i < MAX_NUM_VAL; i++) {
        for (int j = i; j < MAX_NUM_VAL; j += i) {
            num_divisors[j]++;
        }
    }
}

int main() {
    sieve_divisors();

    int count;
    if (scanf("%d", &count) != 1) return 0;

    for (int i = 0; i < count; i++) {
        int chocolate_len;
        scanf("%d", &chocolate_len);
        divisor_tally[num_divisors[chocolate_len]]++;
    }

    long long identical_pairs = 0;
    for (int d = 1; d < 1000; d++) {
        if (divisor_tally[d] > 1) {
            identical_pairs += divisor_tally[d] * (divisor_tally[d] - 1) / 2;
        }
    }

    printf("%lld\n", identical_pairs);
    return 0;
}
```

OUTPUT:

```
10
2 3 4 2 7 6 8 6 4 9
12
```

```
3
2 3 4
1
```